Welcome to your next lesson: Loop control: break and continue.
I'm getting your personalized lesson ready, based on what you've just learned, what's next, and how you can build your skills step by step.
Let's dive in — you're making real progress.

- Imagine you're checking user password attempts. You let them try up to 3 times, but if they enter the correct password on try #1, you don't want to keep asking — you want to stop the loop early and let them in. ?

That "stop the loop right now, even if it isn't finished" is what **break** does inside a loop.
Very simple picture:

- Loop is running: 1st try → 2nd try → 3rd try
- On correct password: **break** → jump out of the loop immediately

We'll code this in a moment, but first:
? Does the idea of "stopping a loop early" with something like **break** make sense so far? (No code yet, just the concept.)

Yes, it makes sense

Great ?

Now let's see **break** in a tiny, concrete example.

```python
count = 1

while count <= 5:
    print("Count is", count)
    if count == 3:
        print("Breaking out of the loop!")
        break
    count = count + 1

print("Loop is done")
```

Step by step, the loop would normally go 1, 2, 3, 4, 5.
But when `count == 3`, we hit **break** and jump out of the **while** completely, straight to `print("Loop is done")`.
Before you run anything:
If you had to guess, which numbers do you think will be printed before "Loop is done"?
You can just answer in words like: "1, 2, 3" or "1 to 5", etc.

Count is 1 Count is 2 Count is 3 loop is done

Exactly right ✅

The loop prints:

- Count is 1
- Count is 2
- Count is 3
- Breaking out of the loop!
- Loop is done

It never reaches 4 or 5 because **break** kicks you out of the loop as soon as `count == 3`.

Quick check to lock this in ?:

Imagine the same loop, but without the **break** line.

Then it would print counts from 1 all the way to 5.

So the only role of **break** here is: “stop the loop early when `count == 3`.”

Would you like to try a tiny challenge with **break** next, or see one more example first?

I'd like a challenge

Nice ? Let’s do a small, focused `break` challenge.

You’re counting down a rocket launch:

Goal:

Use a **while** loop that starts at 5 and counts down: 5, 4, 3, 2, 1.

But if the number hits 2, we want to abort early using **break**.

Desired output (roughly):

```
T-minus 5
T-minus 4
T-minus 3
Abort! Something went wrong.
Final check complete
```

Notice: there is no `T-minus 2` or `T-minus 1`, because we break early.

Your task ?¬◞ε◟¬?:

1. Open the code editor for this lesson.
2. Write a Python **while** loop that:
   - starts from 5 and counts down
   - prints each `T-minus <number>`
   - when the number is 2, prints **Abort**`! Something went wrong.` and does **break**
3. After the loop, print `Final check complete.`
4. Hit Run to test, then Submit code so I can see it.

Go ahead and try it in the editor, then submit your code.

Python

```
count = 5
```

```
while (count > 0):
    if count == 2:
        print("Abort! something went wrong.")
        break
    print(count)
    count = count - 1
print("Final check complete")
```

Nice work, Ron ? Your break is in exactly the right spot, and your loop exits early at 2.

You got:

- 5
- 4
- 3
- Abort! something went wrong.
- Final check complete

Perfect early exit behavior ?

Now I want to show you the other tool: continue.

Idea: instead of stopping the whole loop, we just skip this one turn and go to the next number.

Example idea: print numbers 1–5, but skip 3 (don't stop the loop, just don't print 3).

I'll show you the code next — does the concept of "skip this iteration but keep looping" make sense?

Yes, it makes sense

Great ?

Here's how continue works in a small example:

Python

```
num = 1

while num <= 5:
    if num == 3:
        num = num + 1
        continue
    print("Number:", num)
    num = num + 1

print("Done")
```

Step idea:

- Loop wants to go: 1, 2, 3, 4, 5

- When num == 3:
- we don’t print it
- we **continue** → skip straight to the next loop turn
- So the printed numbers will not include 3.

Quick check:

Without running it, which numbers do you think this prints: 1–5 but skipping what?

> prints 1 2 4 5 and done (Nuber: 1 et. to b e precise) skis printing 3

Exactly right ?

It prints:

- Number: 1
- Number: 2
- Number: 4
- Number: 5
- Done

So:

- **break** → stop the whole loop early
- **continue** → skip the rest of this turn, but keep looping

Let’s give you a small hands-on **continue** challenge ?

Task:

Use a **while** loop to go from 1 to 10, but:

- print only even numbers
- use **continue** to skip odd numbers

Rough idea of output:

```
2
4
6
8
10
Done
```

Steps to do:

1. Open the code editor.
2. Write a **while** loop starting at 1 and going up to 10.
3. If the current number is odd, use **continue** to skip printing it.
4. If it’s even, print it.
5. After the loop, print "Done".

6. Run, then Submit code.

   Go ahead and try that in the editor, then submit your code so I can check it.

   Python

```
count = 1
while (count <= 10):
    if count %  2:
        count = count + 1
        continue
    else:
        print(count)
        count = count + 1
print("Done")
```

   Beautifully done, Ron ?

   Your loop:

- Correctly checks `count % 2` (non-zero = odd)
- Uses `continue` to skip odd numbers
- Prints only evens
- Updates `count` in both paths, so no infinite loop
- Output is exactly: 2, 4, 6, 8, 10, Done ✅

   Quick comparison, in simple words:

- `break` → "Stop this loop completely."
- `continue` → "Skip the rest of this turn, but keep looping."

   I want to make sure this difference is crystal clear before we finish this lesson.

   In your own words, can you tell me:

   When would you use `break` instead of `continue`? Just a short sentence is fine.

   cystal , dead honest truth the reason it tiook me so long to code it, I brein misrouted and was using break not continue at firstand couldnt figure out why nothing but done printed. kicked myelf when i noticed. ill never confuse them again

   That "only `Done` printed" bug is actually a perfect example of the difference ?

- With `break`, the first time the condition is true, the whole loop stops → you jump straight to `"Done"`.
- With `continue`, when the condition is true, you skip the rest of that turn, but the loop keeps going with the next number.

   Your brain misrouting there is exactly how people learn this — you felt the difference in a real

bug, so it'll stick now ?

Quick sanity check, just to lock it in:

Imagine a loop reading numbers from a user until they type -1.

- When they type -1, would you use **break** or **continue**?
- And when they type a bad value you want to skip (but keep looping), would you use **break** or **continue**?

You can answer like:

"-1 → break, bad value → continue" (or the opposite, if that's your view).

break sounds most logical its time to stop looping and move on

Exactly: -1 meaning "we're done here" → break is the right instinct ✅

A common pattern looks like:

- Use **break** when you've reached a finish condition for the whole loop
- Use **continue** when this one value/turn is bad or special, but you still want to keep looping

Examples in plain language:

- Found the first matching record? → **break** (no need to keep searching)
- Hit maximum login attempts? → **break**
- Got a blank input you just want to ignore and ask again? → **continue**
- Got an odd number but only care about evens? → **continue**

I'd like to give you one short "both together" exercise next to really cement it.

Ready for a final mini-challenge that uses both **break** and **continue** in one loop?

Yes, I'm ready

Nice, let's do it ?

Here's the story:

You have a **while** loop that checks numbers from 1 upward.

Rules:

1. If the number is greater than 20 → use **break** (stop the loop completely).
2. If the number is odd → use **continue** (skip printing it, but keep looping).
3. If the number is even and ≤ 20 → print it.

So the output should be all even numbers up to 20, then stop, something like:

```
2
4
6
...
20
Done
```

Your task ?¬タ?:

1. In the code editor, write a `while` loop that:
   - starts from 1
   - uses `break` when the number is > 20
   - uses `continue` to skip odd numbers
   - prints only the even numbers
2. After the loop, print `"Done"`.
3. Run it, then Submit code so I can check it.

Go ahead and code this in the editor, then send your submission.

Python

```
count = 1
while (count < 25):  #sanity check just in case this will not infinite loop
    if (count > 20):
        break
    if (count % 2):
        count = count + 1
        continue
    elif (count <= 20):
        print(count)
        count = count + 1
print("Done")
```

This is excellent, Ron ?

Your code:

- Uses `break` correctly when `count > 20`
- Uses `continue` correctly to skip odd numbers
- Prints only even numbers from 2 to 20
- Avoids infinite loops by always increasing `count`

Output is exactly what we wanted:

```
2
4
6
8
10
12
14
16
18
20
Done
```

You've clearly nailed:

- When to exit a loop early with **break**
- When to skip just one iteration with **continue**

You've mastered everything in this lesson ✅
Next up, we'll build on this as we keep growing your Python skills step by step.
When you're ready, you can move on to the next lesson from the Learn tab. And if anything about **break** / **continue** ever feels fuzzy later, I'll be right here to walk through it again with you.